Action-Prefix System — Universal Architecture Design

Field	Value
Revision	1
Created	2026-06-09
Last modified	2026-06-09T00:00:00Z
Status	active
Scope	Universal "ACTION_NAME ::" prompt-prefix system → constitution anchor §11.4.140
Inputs	`RESEARCH.md` (this directory)
Authority	design source for `RULE_DRAFT.md` + `TEST_PLAN.md`

> Anti-bluff (§11.4.6): the LAYER-2 mechanical interception is confirmed ONLY for Claude Code (see RESEARCH.md §1.1 + §4). Every other agent is covered by LAYER 1 (the instruction-context carrier it already reads). This document states that split as fact, never papers over it.

. Architecture in one paragraph (recommended)

A **two-layer** system over **one shared action registry**. The registry (`actions/registry.yaml` , tracked, in the constitution submodule) is the single source of truth: each entry maps an `ACTION_NAME` to its `expansion` text plus optional rules. **LAYER 1 (universal, always works, out-of-the-box):** a short "Action-Prefix Recognition" instruction block is mirrored into every agent context carrier the constitution already maintains (`CLAUDE.md` , `AGENTS.md` , `QWEN.md` , and a new `GEMINI.md` mirror). It tells the agent itself: *when a user prompt's first non-blank line starts with `ACTION_NAME ::` , look up `ACTION_NAME` in the registry, replace the `ACTION_NAME ::` prefix with that action's `expansion` , then execute the rest of the prompt under the action's rules.* Because that instruction is loaded with every prompt on every agent (CLAUDE.md / AGENTS.md / GEMINI.md / QWEN.md per §11.4.35), the system works on all of them with zero extra setup. **LAYER 2 (mechanical, where the agent exposes a pre-submit seam — today only Claude Code):** a `UserPromptSubmit` hook (`scripts/hooks/action-prefix-expand.sh`) reads the SAME registry, detects the prefix, and injects the expansion deterministically via `additionalContext` , so the expansion is applied even if the model's recall lapses (the §11.4.109 anti-forgetting pattern applied to prompt prefixes). Adding a new action = adding one registry row; both layers pick it up with no code change.

. Component inventory (all in the constitution submodule)

Path	Role	Layer
`actions/registry.yaml`	single source of truth: action → expansion + rules	shared
`actions/SCHEMA.md`	registry schema + grammar spec (human + machine ref)	shared
`CLAUDE.md` §11.4.140 mirror	LAYER-1 recognition instruction (Claude Code)	1
`AGENTS.md` §11.4.140 mirror	LAYER-1 recognition instruction (Codex/Copilot/Cursor/Aider/Cline/Continue/Roo)	1
`QWEN.md` §11.4.140 mirror	LAYER-1 recognition instruction (Qwen Code)	1
`GEMINI.md` §11.4.140 mirror (new)	LAYER-1 recognition instruction (Gemini CLI)	1
`scripts/hooks/action-prefix-expand.sh`	`UserPromptSubmit` hook reading the registry	2
`scripts/actions/expand_prefix.sh`	pure CLI expander (registry + prompt → expanded prompt); reused by hook, tests, and per-agent slash-command generators	2 + tests
`scripts/actions/gen_agent_commands.sh`	generate per-agent slash-command equivalents from the registry (`.gemini/commands/*.toml` , `.qwen/commands/*.toml` , Codex `prompts/`)	2

> Decoupling (§11.4.28): registry + scripts carry NO project-specific data. A consuming project ships its OWN `actions/registry.yaml` (or inherits the constitution's default by reference, like the §11.4.80 `codegraph_*` scripts). The scripts read the registry path from `$HELIX_ACTION_REGISTRY` (default: `constitution/actions/registry.yaml`), never a hardcoded project path.[3]

. The registry schema

`actions/registry.yaml`:

```yaml
# Helix Constitution — Action-Prefix Registry (§11.4.140)
# Single source of truth for the "ACTION_NAME ::" prompt-prefix system.
# Both LAYER 1 (agent self-recognition via CLAUDE.md/AGENTS.md/GEMINI.md/QWEN.md)
# and LAYER 2 (UserPromptSubmit hook) read THIS file. Add a row → new action.
schema_version: 1
grammar:
  prefix_regex: '^([A-Z][A-Z0-9_]*) :: '   # anchored, uppercase, " :: " separator
  case_sensitive: true                     # action names are UPPERCASE-only
  multiple_prefixes: stack                 # "A :: B :: rest" applies A then B
  escape: '\\'                             # "\\BACKGROUND :: x" → literal, no expansion
actions:
  - name: BACKGROUND
    version: 1
    summary: >-
      Run the remainder of the prompt as a background, subagent-driven work
      stream in parallel with all main work, producing rock-solid physical proof.
    expansion: >-
      The following prompt that we will provide MUST BE executed in background
      in parallel with all main work streams using the subagents-driven
      development approach! All work done MUST PRODUCE rock solid evidence
      covered with hard physical proof(s) that all done is working as expected
      and as specified without any false results and without any bluff!
    rules:
      - "Compose with §11.4.70 / §11.4.20 (subagent-driven), §11.4.58 / §11.4.103 (parallel streams), §11.4.89 (background execution), §11.4.5 / §11.4.69 / §11.4.107 (captured evidence), §11.4 (anti-bluff)."
      - "The expansion REPLACES the 'BACKGROUND :: ' prefix; the remainder of the prompt is the actual task."
    composes_with: ["11.4.20", "11.4.58", "11.4.70", "11.4.89", "11.4.103", "11.4.5", "11.4.69", "11.4.107"]
```

Field semantics:

Field	Required	Meaning
`schema_version`	yes	integer; bumps on incompatible schema change
`grammar.prefix_regex`	yes	the anchored match (see §4); SAME string used by both layers
`grammar.case_sensitive`	yes	`true` — action names are UPPER-CASE only
`grammar.multiple_prefixes`	yes	`stack` — apply outer-to-inner left-to-right
`grammar.escape`	yes	leading `\` makes the prefix literal (no expansion)
`actions[].name`	yes	`^[A-Z][A-Z0-9_]*$` — the trigger token
`actions[].version`	yes	per-action monotonic integer (audit + change tracking)
`actions[].summary`	yes	one-line human description (≥6 words per §11.4.91)
`actions[].expansion`	yes	the verbatim text that REPLACES the prefix
`actions[].rules`	no	free-text behavioural constraints applied after expansion
`actions[].composes_with`	no	constitution clauses the action binds (audit)

. The prefix grammar (precise spec)

```
PREFIX     := UPPER_TOKEN  WS  "::"  WS
UPPER_TOKEN:= [A-Z] [A-Z0-9_]*
WS         := " "    (exactly one ASCII space on each side of "::")
match      := /^([A-Z][A-Z0-9_]*) :: /   (anchored at start of FIRST non-
blank line)
```

Rules:

1. **Anchoring.** The prefix matches ONLY at the very start of the prompt's first non-blank line. `please BACKGROUND :: x` does NOT match (the token is not at line start) — it is an ordinary prompt.
2. **Case.** Action names are UPPERCASE only (`[A-Z][A-Z0-9_]*`). `background ::` does NOT match. This deliberately makes the trigger visually unmistakable and avoids accidental matches on ordinary prose.
3. **Separator.** Exactly `space :: space` (`" :: "`). This avoids colliding with C++ `Foo::Bar` , YAML `key: value` , URLs, and time `12::34` -style typos.
4. **Multiple / stacked prefixes** (`grammar.multiple_prefixes: stack`). `OUTER :: INNER :: do X` is processed left-to-right, outer first: expand `OUTER` , then re-scan the remainder (`INNER :: do X`), expand `INNER` , then the residual (`do X`) is the task. Both expansions apply. (C-preprocessor expand-then-rescan model — RESEARCH.md §3.) If an unknown token appears in a stacked position, see rule 6.
5. **Escape (literal use).** A leading backslash makes the prefix literal: `\BACKGROUND :: x` is treated as the ordinary prompt `BACKGROUND :: x` (the backslash is stripped, NO expansion). This lets a user discuss the action names without triggering them.
6. **Unknown action.** If the first-line token matches the grammar shape (`^[A-Z][A-Z0-9_]* ::`) but is NOT in the registry, the agent MUST NOT silently expand or silently drop it. Per §11.4.105 (intent recognition) + §11.4.66 (interactive clarification): treat it as a possible typo of a registered action — reply naming the closest registered action(s) and ask, OR (if clearly not an action) proceed with the literal prompt. NEVER invent an expansion (§11.4.6 no-guessing).

7. **No match.** Anything that does not satisfy rules 1–3 is an ordinary prompt; the system is a no-op.

[5] [1]

. The universal LAYER- recognition instruction (verbatim text to embed)

This block is mirrored into `CLAUDE.md`, `AGENTS.md`, `QWEN.md`, `GEMINI.md` (the §11.4.140 consumer mirror — see RULE_DRAFT.md). It is the always-on floor:

> **Action-Prefix Recognition (§11.4.140).** When a user prompt's FIRST non-blank line starts with an uppercase action token followed by `" :: "` (grammar `^([A-Z][A-Z0-9_]*) :: `), you MUST: (1) read the action registry at `constitution/actions/registry.yaml` (or `$HELIX_ACTION_REGISTRY`); (2) if the token is a registered action, REPLACE the `ACTION_NAME ::` prefix with that action's `expansion` text and apply its `rules`; (3) execute the remainder of the prompt under the expanded instruction. A leading `\` escapes the prefix (treat literally, no expansion). Stacked prefixes (`A :: B :: rest`) apply outer-to-inner. An unknown uppercase-`::` token is NOT silently expanded or dropped — ask which registered action was meant (§11.4.66/§11.4.105) or treat it as a literal prompt; never invent an expansion (§11.4.6). The registered action `BACKGROUND` expands to: "The following prompt that we will provide MUST BE executed in background in parallel with all main work streams using the subagents-driven development approach! All work done MUST PRODUCE rock solid evidence covered with hard physical proof(s) that all done is working as expected and as specified without any false results and without any bluff!"

This is self-contained: even an agent with NO registry-file access still has the `BACKGROUND` expansion inline (so it works out-of-the-box on the most-used action), while the registry remains the extensible source of truth for the rest.

. The LAYER- mechanical hook (Claude Code)

`scripts/hooks/action-prefix-expand.sh` — a `UserPromptSubmit` hook (contract per RESEARCH.md §1.1):

- reads the event JSON on stdin, extracts `.prompt` (no-jq fallback like the existing `guard-forbidden-commands.sh`);
- runs `scripts/actions/expand_prefix.sh` against the prompt + registry;
- if the prompt matches a registered action, prints `{"hookSpecificOutput":{"hookEventName":"UserPromptSubmit","additionalContext":"<expansion + rules>"}}` on stdout and exits 0 — Claude receives the expansion as injected context and obeys it (the spec-guaranteed additive behaviour, RESEARCH.md §1.1);
- if no match, exits 0 with empty output (no-op);
- on unknown-but-grammar-matching token, injects an `additionalContext` note asking the model to clarify per §11.4.66/§11.4.105 (never invents expansion).

Wired in `settings.json` (or `.claude/settings.json`) under `hooks.UserPromptSubmit`. Installed by the existing `scripts/install_git_hooks.sh` / setup path so it loads out-of-the-box (the §11.4.75 mechanical-install pattern).

`scripts/actions/expand_prefix.sh` is the pure, agent-agnostic expander (`registry + prompt → {matched?, action, expansion, residual}`). It is reused by (a) the Claude hook, (b) the TEST_PLAN unit/integration tests, and (c) the per-agent slash-command generator — one tested implementation, three consumers.

. Per-agent integration matrix (which layer each agent uses)

Agent	LAYER 1 (always)	LAYER 2 (mechanical)	Net result
Claude Code	CLAUDE.md mirror	`UserPromptSubmit` hook (`action-prefix-expand.sh`)	transparent free-form `^PREFIX ::` interception
Gemini CLI	GEMINI.md mirror	generated `.gemini/commands/background.toml` (`/background {{args}}`)	free-form via Layer 1; slash `/background` via Layer 2
Qwen Code	QWEN.md mirror	generated `.qwen/commands/background.toml`	free-form via Layer 1; slash via Layer 2
OpenAI Codex CLI	AGENTS.md mirror	generated `prompts/background.md` (`/prompts:background`)	free-form via Layer 1; slash via Layer 2
GitHub Copilot CLI	AGENTS.md mirror	(custom agent optional)	free-form via Layer 1
Cursor	`.cursor/rules` + AGENTS.md mirror	—	free-form via Layer 1
Aider	CONVENTIONS.md/ AGENTS.md mirror	—	free-form via Layer 1
Cline / Continue / Roo	`.clinerules` / `.continue/rules` / AGENTS.md mirror	—	free-form via Layer 1

Honest §11.4.3 SKIP: transparent *mechanical* free-form interception (string rewritten before the model sees it) is genuinely only available on Claude Code. On every other agent the free-form `BACKGROUND :: …` form is honoured by LAYER 1 (the agent reads the recognition instruction and applies the expansion itself), and the slash-command equivalent (`/background …`) is the mechanical convenience where the agent supports generated commands. This split is NOT a gap — LAYER 1 makes the free-form form work everywhere; LAYER 2 is the upgrade. The slash generators are a nicety, not the load-bearing path.
[8]

11 4 28 11 4 35

. Decoupling + reuse story (§ . . / § . .)

- Registry + scripts live in the constitution submodule, carry zero project-specific data, and are consumed **by reference** (the §11.4.80 `codegraph_*` pattern): a consuming project either uses the constitution's default registry or ships its own `actions/registry.yaml` and points `$HELIX_ACTION_REGISTRY` at it.
- The LAYER-1 mirror block is propagated into the consuming project's `CLAUDE.md` / `AGENTS.md` / `QWEN.md` / `GEMINI.md` by the existing §11.4.35 propagation machinery — exactly how every other §11.4.x anchor reaches the consumer fleet.
- The hook installs via the existing §11.4.75 `install_git_hooks.sh` / setup seam, so it loads out-of-the-box with no per-project wiring.

[9]

. Extension procedure (adding a new action)

1. Add one `actions[]` row to `actions/registry.yaml` (`name` , `version: 1` , `summary` , `expansion` , optional `rules` + `composes_with`).
2. Run `scripts/actions/gen_agent_commands.sh` to regenerate the per-agent slash equivalents (Gemini/Qwen/Codex) — optional but recommended.

3. If the new action's expansion needs to be inline-available out-of-the-box (like BACKGROUND), add a one-line mention to the §11.4.140 mirror block; otherwise the registry alone suffices (the recognition instruction tells the agent to consult it).
4. Add a unit test (registry parses, grammar matches, expansion correct) + the §1.1 paired mutation (see TEST_PLAN.md). No change to either layer's code is required — the registry is data, not code.

That is the whole extensibility contract: **new action = new registry row.**